

LEARNING ROADMAP

# Building LLM-Powered RAG Applications

Production-Ready Retrieval Systems with Java & Spring AI

**Intensive**

40-50 hrs/week

**Balanced**

15-20 hrs/week

**Flexible**

8-10 hrs/week

■ 5 Phases      ★ Intermediate to Advanced

## Overview

This roadmap teaches you to build production-ready Retrieval-Augmented Generation (RAG) systems using Java 21 and Spring Boot. Unlike typical Python-focused resources, this path leverages the mature Java ecosystem with Spring AI and LangChain4j frameworks. You'll start with LLM foundations, master embeddings and vector search, build complete RAG pipelines, implement production guardrails, and deploy with enterprise-grade observability. Based on Crio.do's intensive workshop curriculum, expanded into a comprehensive learning journey.

### Who is this for?

Java/Spring Boot developers wanting to add AI/LLM skills, backend engineers transitioning to AI engineering roles, enterprise developers building internal RAG systems, or developers from Python AI backgrounds seeking Java proficiency.

### Prerequisites

- Solid Java fundamentals (OOP, Collections, Streams, Generics)
- Spring Boot basics (IoC, DI, REST APIs, Spring Data JPA)
- Basic SQL and database concepts
- Git version control proficiency
- Understanding of HTTP and REST principles
- Docker basics (helpful but not required)

### What you'll learn

- Understand LLM architecture and how to interact with AI APIs from Java
- Work with embeddings and vector databases for semantic search
- Design and build end-to-end RAG system architecture
- Implement document ingestion pipelines for multiple formats
- Master chunking strategies and retrieval optimization
- Build production guardrails: citations, refusals, and safety
- Deploy RAG systems with Docker and implement observability

- Evaluate and test LLM applications systematically
- Use both Spring AI and LangChain4j frameworks effectively

## Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

### Intensive

Fast Track

**40-50**

hrs/week

**280-**

**350**

total hrs

Full-time immersive learning. Ideal for career changers or those with dedicated time to rapidly transition into AI engineering.

#### Ideal for:

- Career changers with dedicated time
- Developers between jobs
- Those preparing for AI engineer interviews
- Bootcamp-style learners

### Balanced

Recommended

**15-20**

hrs/week

**180-**

**240**

total hrs

Balanced pace for working professionals. Deep understanding while maintaining job commitments. The recommended path for most learners.

#### Ideal for:

- Working Java developers adding AI skills
- Software engineers transitioning to AI
- Those who prefer thorough mastery
- Team leads evaluating RAG for their organization

### Flexible

Self-paced

**8-10**

hrs/week

**192-**

**240**

total hrs

Flexible learning for those with limited availability. More time to absorb complex concepts and build portfolio projects.

#### Ideal for:

- Full-time employees with busy schedules
- Parents and caregivers
- Those learning multiple skills
- Weekend and evening learners

## Phase Schedule by Plan

| Phase | Topic                                  | Intensive | Balanced    | Flexible    |
|-------|----------------------------------------|-----------|-------------|-------------|
| 1     | LLM Foundations & Java AI Ecosystem    | Week 1    | Weeks 1-2   | Weeks 1-4   |
| 2     | Embeddings & Vector Search             | Week 2    | Weeks 3-4   | Weeks 5-8   |
| 3     | RAG System Architecture                | Weeks 3-4 | Weeks 5-7   | Weeks 9-14  |
| 4     | Production Hardening: Safety & Quality | Week 5    | Weeks 8-9   | Weeks 15-18 |
| 5     | Deployment, Testing & Evaluation       | Weeks 6-7 | Weeks 10-12 | Weeks 19-24 |

## 1

# LLM Foundations & Java AI Ecosystem

Intensive: Week 1

Balanced: Weeks 1-2

Flexible: Weeks 1-4

Before diving into RAG, you must understand what LLMs are, how to interact with them, and the Java ecosystem that makes this possible. Unlike Python where LangChain dominates, Java offers two mature frameworks: Spring AI and LangChain4j. Understanding both gives you flexibility and makes you versatile in the job market.

## Learning Objectives

- Understand transformer architecture at a conceptual level
- Interact with LLM APIs (OpenAI, Anthropic, Ollama) from Java
- Set up Spring AI and LangChain4j projects
- Master prompt engineering fundamentals
- Understand token economics and context windows
- Compare Spring AI vs LangChain4j approaches

## Topics to Cover

### Understanding Large Language Models

You cannot effectively build with LLMs if you treat them as black boxes. Understanding how they work helps you debug issues and craft better prompts.

- Transformer architecture intuition
- Autoregressive text generation
- Pre-training, fine-tuning, and RLHF
- Tokenization and vocabulary
- Context windows and their constraints
- Model capabilities and limitations

### Java AI Framework Setup

Spring AI and LangChain4j are your primary tools. Setting them up correctly saves hours of debugging later.

- Spring AI with Spring Initializr
- LangChain4j Maven/Gradle setup
- API key configuration
- Model provider setup (OpenAI, Anthropic, Ollama)
- Auto-configuration in Spring AI
- Framework comparison: when to use which

## LLM API Integration

Direct API interaction is the foundation. Before abstractions, understand what happens at the HTTP level.

- ChatClient and ChatModel (Spring AI)
- ChatLanguageModel (LangChain4j)
- Synchronous vs streaming responses
- Error handling and retry strategies
- Rate limiting considerations
- Local model support with Ollama

## Prompt Engineering Fundamentals

The quality of your RAG system depends heavily on prompt design. Good prompts reduce hallucinations and improve answer quality.

- Zero-shot and few-shot prompting
- Chain-of-thought reasoning
- System prompts and personas
- Prompt templates in both frameworks
- Structured output (JSON mode)
- Output parsers and validators

## Hands-on Projects

### Multi-Provider Chat Application

Build a Spring Boot application that can switch between OpenAI, Anthropic, and local Ollama models via configuration. Implement streaming responses and a simple web interface.

Spring AI

Provider Configuration

Streaming

WebFlux

### Prompt Engineering Lab

Create a prompt testing harness that evaluates different prompt strategies for summarization and Q&A tasks. Compare outputs and token usage across approaches.

Prompt Design

Token Economics

Evaluation

LangChain4j

## Resources

DOCUMENTATION

[Spring AI Documentation](#)

DOCUMENTATION

[LangChain4j Documentation](#)

TOOL

[Ollama - Run LLMs Locally](#)

### Milestone



Build and demonstrate a working chat application in both Spring AI and LangChain4j. Explain the tradeoffs between frameworks. Write effective prompts that consistently produce structured outputs.

## 2 Embeddings & Vector Search

Intensive: Week 2

Balanced: Weeks 3-4

Flexible: Weeks 5-8

Embeddings are the heart of semantic search and RAG. This phase teaches you how text becomes vectors, how similarity works in high-dimensional space, and how vector databases enable efficient retrieval. You'll work with multiple vector stores to understand their tradeoffs—essential knowledge for building retrieval pipelines.

### Learning Objectives

- Understand what embeddings are and how they capture meaning
- Work with embedding models (OpenAI, Cohere, local models)
- Set up and interact with vector databases from Java
- Implement similarity search with metadata filtering
- Understand indexing algorithms (HNSW, IVF) conceptually
- Compare vector database options for Java applications

### Topics to Cover

#### Understanding Embeddings

Embeddings convert text into numerical representations that capture semantic meaning. Understanding this enables you to debug retrieval issues.

- What embeddings are and why they matter
- How embedding models work
- Embedding dimensions and implications
- Cosine similarity and distance metrics
- Embedding model comparison
- EmbeddingModel in Spring AI and LangChain4j

#### Vector Database Fundamentals

Vector databases are purpose-built for similarity search at scale. Knowing their architecture helps you optimize performance.

- Why traditional databases fail for vector search
- Approximate Nearest Neighbor (ANN) algorithms
- HNSW index explained
- Metadata filtering in vector search
- Consistency vs performance tradeoffs
- Choosing the right vector DB

#### Spring AI VectorStore Integration

Spring AI provides a unified VectorStore interface enabling portable code across different backends.

- VectorStore interface
- PgVector setup and configuration
- Milvus Java client integration
- Weaviate with Spring AI
- Qdrant integration
- SimpleVectorStore for development

#### LangChain4j EmbeddingStore

LangChain4j offers its own embedding store abstraction with different design choices.

- EmbeddingStore interface
- ContentRetriever abstraction
- Supported stores (Milvus, Pinecone, Qdrant)
- Metadata and filtering
- Hybrid search capabilities

## Hands-on Projects

### Semantic Search Engine

Build a semantic search application over a document corpus (product catalog or FAQ). Implement both keyword and semantic search, compare results. Use PgVector as the backend.

[Embeddings](#)[PgVector](#)[Search Comparison](#)[Spring Data](#)

### Multi-Vector-Store Benchmark

Implement the same search functionality across three vector stores (PgVector, Milvus, Qdrant). Benchmark insertion speed, query latency, and filtered search. Document tradeoffs.

[Vector DBs](#)[Benchmarking](#)[Docker](#)[Performance Analysis](#)

## Resources

[DOCUMENTATION](#)[PgVector Documentation](#)[DOCUMENTATION](#)[Milvus Java SDK](#)[DOCUMENTATION](#)[Spring AI Vector Store Docs](#)

### Milestone



Deploy a semantic search application backed by PgVector. Demonstrate understanding of embedding spaces by visualizing similar documents. Explain when to use different vector databases.

# 3 RAG System Architecture

Intensive: Weeks 3-4

Balanced: Weeks 5-7

Flexible: Weeks 9-14

This is the core RAG phase where you build complete retrieval-augmented generation pipelines. This phase directly incorporates three modules from the Crio.do workshop: System Architecture & Corpus Ingestion (Module 1), Chunking/Embeddings/Index Design (Module 2), and Query Pipeline/Retrieval/Grounded Prompting (Module 3). By the end, you'll have a working production-style RAG system.

## Learning Objectives

- Design end-to-end RAG system architecture
- Implement document ingestion pipelines for multiple formats
- Master chunking strategies and their tradeoffs
- Build efficient retrieval pipelines with re-ranking
- Implement grounded prompting with retrieved context
- Use the Advisor pattern in Spring AI

## Topics to Cover

### RAG System Architecture (Workshop Module 1)

Before coding, understand the big picture. RAG systems have distinct components that must work together. Poor architecture leads to poor results.

- RAG system components overview
- Offline (indexing) vs online (query) pipelines
- Data flow from documents to answers
- Architecture patterns (naive, advanced, modular RAG)
- Corpus requirements and data quality
- Spring AI ETL pipeline overview

### Document Ingestion & Processing (Workshop Module 1)

Garbage in, garbage out. How you ingest documents determines retrieval quality.

- DocumentReader implementations (PDF, HTML, JSON, TXT)
- Apache Tika integration
- Metadata extraction and enrichment
- Handling tables and structured content
- Document preprocessing and cleaning
- Batch ingestion patterns

## Chunking Strategies (Workshop Module 2)

Chunking is critical. Too large and context is diluted. Too small and meaning is lost. Understanding chunking strategies improves retrieval relevance.

- Why chunking matters for LLMs
- Fixed-size chunking with overlap
- Semantic chunking (paragraph, section)
- Recursive character text splitting
- Parent-child chunking strategies
- DocumentTransformer in Spring AI

## Index Design & Embedding Strategy (Workshop Module 2)

How you index affects retrieval performance. Understanding index design helps optimize for your use case.

- Choosing embedding models for your domain
- Index configuration (HNSW parameters)
- Metadata schema design
- Multi-tenancy considerations
- Index refresh and update strategies
- Embedding batch processing

## Query Pipeline & Retrieval (Workshop Module 3)

The query pipeline transforms user questions into relevant context. Multiple retrieval strategies exist.

- Query transformation techniques
- HyDE (Hypothetical Document Embeddings)
- Multi-query retrieval
- Maximal Marginal Relevance (MMR)
- Hybrid search (vector + keyword)
- Re-ranking retrieved results

## Grounded Prompting (Workshop Module 3)

Grounding LLM responses in retrieved context reduces hallucinations. This is the key technique that makes RAG work.

- Context injection strategies
- System prompts for RAG
- QuestionAnswerAdvisor in Spring AI
- Handling contradictory sources
- Long context management
- Chat memory integration

## Hands-on Projects

### Document Q&A System

Build a complete RAG system that answers questions about uploaded documents (PDFs, markdown, HTML). Implement ingestion, chunking, embedding, storage, retrieval, and answer generation with source attribution.

Full RAG Pipeline

Spring AI

Document Processing

PgVector

### Enterprise Knowledge Base

Create a multi-user knowledge base with separate document collections per team. Implement hybrid search, metadata filtering, and conversation history.

Multi-tenancy

Hybrid Search

LangChain4j

Chat Memory

### Technical Documentation Assistant

Build a RAG system over API docs and user guides. Implement code snippet handling, version-aware retrieval, and structured answers.

Specialized Chunking

Code Handling

Version Management

## Resources

DOCUMENTATION [Spring AI Advisors Documentation](#)

TUTORIAL [Building RAG with Spring AI and MongoDB](#)

TUTORIAL [RAG with Spring AI Tutorial](#)

### Milestone



Deploy a working RAG system that answers questions about a document corpus with cited sources. Demonstrate understanding of chunking tradeoffs. Compare retrieval quality across different strategies.

# 4

## Production Hardening: Safety & Quality

Intensive: Week 5

Balanced: Weeks 8-9

Flexible: Weeks 15-18

RAG systems in production must handle edge cases gracefully, refuse inappropriate requests, cite sources accurately, and manage costs. This phase covers the 'production-ready' aspects that distinguish toy projects from enterprise systems. It incorporates Workshop Modules 4 (Citations, Refusal & Guardrails) and 5 (Cost, Observability & Deployment).

### Learning Objectives

- Implement accurate citation and source attribution
- Build refusal mechanisms for unanswerable queries
- Set up input/output guardrails for safety
- Manage LLM costs effectively
- Implement observability for RAG systems
- Apply security best practices

### Topics to Cover

#### Citations & Source Attribution (Workshop Module 4)

Users need to verify answers. Accurate citations build trust and enable fact-checking. This is table stakes for enterprise RAG.

- Citation formats and standards
- Tracking sources through the pipeline
- Passage-level vs document-level citations
- Citation extraction implementation
- Verification of citations
- UI/UX for displaying citations

#### Refusal Mechanisms (Workshop Module 4)

A good RAG system knows when it doesn't know. Refusing with low-confidence results is better than hallucinating.

- Detecting low-confidence retrievals
- Confidence scoring strategies
- Graceful refusal prompts
- Fallback strategies
- 'I don't know' as valid response
- User feedback loops

### Guardrails & Safety (Workshop Module 4)

LLMs can be jailbroken, generate harmful content, or leak data. Guardrails protect users and organizations.

- Input guardrails (prompt injection detection)
- Output guardrails (content filtering)
- PII detection and redaction
- Topic restriction enforcement
- Guardrails in LangChain4j
- Moderation APIs

### Cost Management (Workshop Module 5)

LLM APIs are expensive at scale. Token optimization directly impacts your bottom line.

- Token counting and estimation
- Cost tracking per query
- Caching strategies (semantic caching)
- Model selection by task complexity
- Context window optimization
- Embedding model cost comparison

### Observability & Monitoring (Workshop Module 5)

You cannot improve what you cannot measure. Observability reveals retrieval quality issues and cost anomalies.

- Structured logging for RAG pipelines
- Latency tracking by component
- Token usage monitoring
- Retrieval quality metrics (MRR, Recall@K)
- Spring Boot Actuator for health
- Prometheus and Grafana integration

### Security Considerations

RAG systems handle sensitive documents and queries. Security must be designed in, not bolted on.

- API key management (Vault, secrets)
- Document access control
- Query logging and audit trails
- Data residency requirements
- Prompt injection prevention
- Rate limiting and abuse prevention

## Hands-on Projects

### Production-Ready RAG Service

Harden your Phase 3 RAG system for production. Add citation extraction, confidence scoring, refusal mechanisms, guardrails, cost tracking, and observability. Deploy to cloud.

Guardrails

Observability

Spring Actuator

Security

### RAG Admin Dashboard

Build an admin interface displaying usage metrics, cost breakdown, latency distributions, and retrieval quality. Include alerting for anomalies.

Monitoring

Grafana/Prometheus

Cost Analysis

Admin UI

## Resources

DOCUMENTATION

[LangChain4j Guardrails](#)

DOCUMENTATION

[Spring Boot Actuator](#)

DOCUMENTATION

[Micrometer Metrics](#)

### Milestone



Demonstrate a RAG system with working guardrails, accurate citations, and a monitoring dashboard. Show cost-per-query tracking and explain optimization strategies at scale.

## 5

## Deployment, Testing & Evaluation

Intensive: Weeks 6-7

Balanced: Weeks 10-12

Flexible: Weeks 19-24

The final phase focuses on testing, evaluation, deployment, and continuous improvement. Workshop Module 6 (Break-It & Evaluation Lab) is central here. You'll learn to systematically evaluate RAG systems, deploy them reliably, and establish processes for ongoing quality assurance. This phase transforms you from a developer into an AI engineer.

### Learning Objectives

- Design and run RAG evaluation frameworks
- Write unit and integration tests for LLM applications
- Deploy RAG systems with Docker and Kubernetes
- Implement CI/CD for AI applications
- Conduct adversarial testing (break-it methodology)
- Plan for continuous improvement

### Topics to Cover

#### RAG Evaluation Framework (Workshop Module 6)

Vibes-based evaluation doesn't scale. Systematic evaluation reveals whether changes improve or degrade quality.

- Evaluation dimensions (relevance, faithfulness)
- Creating evaluation datasets
- Ground truth annotation
- Retrieval metrics (Recall@K, MRR, NDCG)
- Generation evaluation (ROUGE, BERTScore)
- LLM-as-judge evaluation
- A/B testing for RAG

#### Testing LLM Applications

Traditional testing doesn't cover LLM non-determinism. You need new strategies for testing AI applications.

- Unit testing with mocked LLM responses
- Integration testing RAG pipelines
- Snapshot testing for prompt changes
- Property-based testing for embeddings
- Contract testing for AI services
- TestContainers for vector DBs

## Adversarial Testing - Break-It Lab (Workshop Module 6)

Real users will try to break your system. Better you find vulnerabilities first.

- Prompt injection attacks
- Retrieval poisoning scenarios
- Edge case identification
- Stress testing with malformed input
- Jailbreak attempts
- Documenting and fixing vulnerabilities

## Containerization & Deployment

Docker ensures consistent environments. Kubernetes enables scaling. Prerequisites for production deployment.

- Dockerfile for Spring Boot AI apps
- Multi-stage builds
- Docker Compose with vector DB
- Environment configuration
- Kubernetes deployment basics
- Health checks and readiness probes

## CI/CD for AI Applications

Continuous integration catches regressions. AI apps need adapted CI/CD practices.

- GitHub Actions for AI projects
- Running evaluations in CI
- Model versioning strategies
- Rollback strategies
- Feature flags for AI features
- Canary deployments

## Continuous Improvement

RAG systems degrade over time as documents change. Continuous improvement keeps quality high.

- Feedback collection mechanisms
- Retrieval quality monitoring
- Re-indexing strategies
- Model upgrade evaluation
- A/B testing infrastructure
- Documentation and runbooks

## Hands-on Projects

### RAG Evaluation Suite

Build a comprehensive evaluation framework for your RAG system. Create a test dataset with ground truth, implement multiple metrics, and set up automated evaluation in CI.

[Evaluation Design](#)[Testing](#)[CI/CD](#)[Metrics](#)

### Break-It Challenge (Workshop Module 6)

Systematically attack your RAG system. Document at least 10 vulnerability types, attempt exploitation, and implement fixes. Create a security report.

[Security Testing](#)[Adversarial Thinking](#)[Documentation](#)

### Production Deployment

Deploy your complete RAG system to cloud. Implement CI/CD, monitoring, alerting, and auto-scaling. Document architecture and operations runbook.

[Docker](#)[Kubernetes](#)[Cloud Deployment](#)[DevOps](#)

## Resources

[DOCUMENTATION](#)[RAGAS Evaluation Framework](#)

DOCUMENTATION

TestContainers for Java

DOCUMENTATION

Docker Documentation

TUTORIAL

Kubernetes Basics

### Milestone



Deploy a production-ready RAG system with automated testing, evaluation in CI, and monitoring. Complete the break-it challenge with documented fixes. Create operations documentation.

Phase 5

## Your Path Forward

### Career Opportunities

✓ AI Engineer

✓ LLM/RAG Developer

✓ AI Platform Engineer

✓ Senior Backend Developer  
(AI focus)

✓ Solutions Architect (AI)

✓ ML Engineer

✓ AI Product Engineer

### Tips for Success

- Don't skip the foundations – understanding LLMs conceptually prevents debugging nightmares later
- Build everything twice – once with Spring AI, once with LangChain4j to understand tradeoffs
- Use real documents – toy datasets hide chunking and retrieval issues that appear in production
- Measure everything – token costs, latency, and retrieval quality should be dashboarded from day one
- The Java AI ecosystem is maturing rapidly – follow Spring AI and LangChain4j releases monthly
- Local models matter – Ollama lets you develop without API costs; always test locally first
- Guardrails are not optional – production systems without guardrails will be exploited
- Documentation is a feature – your RAG system is only as good as the documents you index
- Join the communities – Spring AI Discord, LangChain4j GitHub discussions
- Keep a learning journal – document what works and fails; this becomes your runbook

### Next Steps

1. Ensure Java 21 and Docker are installed
2. Create accounts for OpenAI and Anthropic APIs
3. Set up IntelliJ IDEA with Spring Boot plugins
4. Clone the Spring AI samples repository
5. Choose your learning pace (intensive, balanced, or flexible)
6. Start with Phase 1, Week 1 – do not skip ahead
7. Join Crio.Do community for support and accountability



## Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support – we'd love to have you at Crio.Do

**Explore Crio.Do**

<https://crio.do>